

Algorithms, AI & Data Science II

Ingo Scholtes

Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg
ingo.scholtes@uni-wuerzburg.de

Notes:

- **L07: Logical Inference** 21.05.2025
- **Educational objective:** We introduce logical inference. We show how we can use rules of inference to infer logical conclusions based on a knowledgebase in propositional logic and introduce
 - Logical Inference, Reasoning, and Deduction
 - Rules of Inference in Propositional Logic
 - Forward Chaining and Resolution
- **Handout version:** 2025/05/19 13:02:21

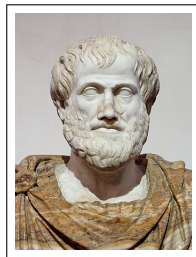
Motivation

- ▶ we can use formal logic to **represent knowledge**
- ▶ hallmark of “intelligence” is ability to use knowledge to **create new conclusions**

[Intelligence is a] mental capability that [...] involves the ability to reason, plan, solve problems, think abstractly, comprehend complex ideas, learn quickly and learn from experience. [...] a capability for comprehending our surroundings—“catching on”, “making sense” of things, or “figuring out” what to do.”

→ Mainstream Science on Intelligence, The Wall Street Journal, 1994

- ▶ **inference** refers to the process of drawing conclusions based on prior knowledge
- ▶ basis for **decision-making** and **problem-solving**
- ▶ how can we **use formal logic to infer conclusions?**



Aristoteles, 384 – 322 BC

image credit: Wikimedia Commons, CC-BY-SA

exemplary knowledgebase (in propositional logic)

- ▶ It rains. (X_0)
- ▶ I have no umbrella. ($\neg X_1$)
- ▶ If it rains and I have no umbrella, then I am wet.
($X_0 \wedge \neg X_1 \implies X_2$)

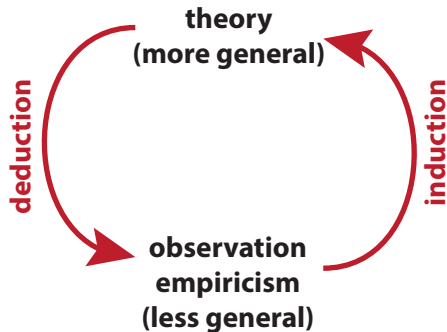
Notes:

- The term “inference” comes from Latin “infero” (to bring forward, to introduce, to cause, to produce, to conclude)

Deductive vs. inductive inference

- ▶ **inductive inference**: we use observations to derive general principles
 - ▶ e.g. infer laws of gravity based on experiments
 - ▶ induco = to introduce, to lead in, to establish
 - ▶ example: statistical inference → AKIDS 2, Part II
- ▶ **deductive inference**: we derive specific conclusion based on general theory
 - ▶ e.g. infer trajectory based on laws of gravity
 - ▶ deduco = to lead off, to subtract, to reduce
 - ▶ example: logical inference → today and next week
- ▶ using (deductive) logical inference we can **derive valid conclusions from premises**

A deductive inference is **valid** iff the truth of all premises ensures the truth of the conclusions.



example

- ▶ Premise 1: It rains. (X_0)
- ▶ Premise 2: I have no umbrella. ($\neg X_1$)
- ▶ Conclusion: I am wet. (X_2)

Notes:

Rules of inference

- ▶ a **rule of inference** is a transformation rule in formal logics which, starting from one more **premises** P_i , allows to infer a **conclusion** C
- ▶ rule of inference is **valid** iff $P_1 \wedge \dots \wedge P_n \implies C$ is tautology, i.e. **truth of premises implies conclusion**
- ▶ for premises P_i and conclusion C we denote rules of inference in **standard form**

rule of inference

$$\frac{P_1 \quad P_2 \quad \dots \quad P_n}{C}$$

- ▶ we can use valid rules of inference to **transform expressions in propositional logic**

example: And-Elimination

$$\frac{X_1 \wedge X_2}{X_1}$$

$$\frac{X_1 \wedge X_2}{X_2}$$

X_1	X_2	$X_1 \wedge X_2$	$X_1 \wedge X_2 \implies X_1$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	1

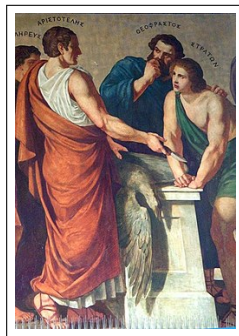
Notes:

Modus ponens

- ▶ modus ponens (“mode that affirms”) is rule of inference that is basis for **direct proofs**
- ▶ correctness of the **consequent** is based on correctness of **antecedent** and implication
- ▶ most commonly used form of logical reasoning
- ▶ we can use truth table to prove **underlying tautology**

X_0	X_1	$X_0 \implies X_1$	$X_0 \wedge (X_0 \implies X_1) \implies X_1$
0	0	1	1
0	1	1	1
1	0	0	1
1	1	1	1

- ▶ rule of inference with two premises and one conclusion is called **syllogism** ($\sigma\upsilon\lambda\lambda\omicron\gamma\iota\sigma\mu\omicron\zeta$ = conclusion)



Theophrastus, 371 BC – 287 BC

image credit: Wikimedia Commons, Carl Rahl, c. 1888, public domain

Modus ponens

$$\frac{X_0 \implies X_1 \quad X_0}{X_1}$$

Notes:

Modus tollens

- ▶ modus tollens (“mode that denies”) is a syllogism that is basis for **falsification of statements**
- ▶ falsification of premise is based on implication and fact that conclusion is false, i.e. if antecedent was true then consequent would be true
- ▶ we can use truth table to prove **underlying tautology**

X_0	X_1	$X_0 \implies X_1$	$\neg X_1 \wedge (X_0 \implies X_1) \implies \neg X_0$
0	0	1	1
0	1	1	1
1	0	0	1
1	1	1	1

- ▶ note that from $X_0 \implies X_1$ and $\neg X_1$ and we can **not** conclude $\neg X_0$ → fallacy of the inverse

Modus tollens

$$\begin{array}{r} X_0 \implies X_1 \\ \neg X_1 \\ \hline \neg X_0 \end{array}$$

modus tollens

“If there is a burglar in the house, the dog will bark. Since the dog does not bark, there is no burglar in the house.”

fallacy of the inverse

“If the dog barks, there is a burglar in the house. Since the dog does not bark, there is no burglar in the house.”

Notes:

Hypothetical syllogism

- ▶ hypothetical syllogism uses **transitive closure of logical implication** to derive new conclusions
- ▶ requires two implications, where the consequent of the first is the antecedent of the second
- ▶ we can use truth table to prove **underlying tautology**

Hypothetical syllogism

$$\begin{array}{l} X_0 \implies X_1 \\ X_1 \implies X_2 \\ \hline X_0 \implies X_2 \end{array}$$

example

"If there is a burglar in the house, the dog will bark. If the dog barks I will wake-up. Hence, if there is a burglar in the house I will wake up."

X_0	X_1	X_2	$A := (X_0 \implies X_1)$	$B := (X_1 \implies X_2)$	$A \wedge B \implies (X_0 \implies X_2)$
0	0	0	1	1	1
0	0	1	1	1	1
0	1	0	1	0	1
0	1	1	1	1	1
1	0	0	0	1	1
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	1	1	1

Notes:

Disjunctive syllogism

- ▶ disjunctive syllogism or **modus tollendo ponens** (“mode that affirms by denying”) uses fact that, for a disjunction to be true, at least one of the propositions must be true
- ▶ we can use a truth table to prove **underlying tautology**

X_0	X_1	$X_0 \vee X_1$	$(X_0 \vee X_1) \wedge \neg X_0$	$(X_0 \vee X_1) \wedge \neg X_0 \implies X_1$
0	0	0	0	1
0	1	1	1	1
1	0	1	0	1
1	1	1	0	1

- ▶ allows to **eliminate disjunctions** for logical inference in propositional logic
- ▶ using De Morgan’s law, we can derive **logically equivalent** rule for conjunction → self-study questions

Disjunctive syllogism

$$\frac{X_0 \vee X_1 \quad \neg X_0}{X_1}$$

example 1

“I stay at home or go to a party. I am not at home, hence I am at a party.”

Conjunctive syllogism

$$\frac{\neg(X_0 \wedge X_1) \quad X_0}{\neg X_1}$$

example 2

“I cannot stay at home and go to a party. I am at home, hence I am not at a party.”

Notes:

Contraposition

- ▶ we can flip direction of implication iff we negate antecedent and consequent
- ▶ we can use a truth table to prove **underlying tautology**

		$A :=$	$B :=$	
X_0	X_1	$X_0 \implies X_1$	$\neg X_1 \implies \neg X_0$	$A \iff B$
0	0	1	1	1
0	1	1	1	1
1	0	0	0	1
1	1	1	1	1

- ▶ different from other rules of inference, in contraposition premise and conclusion are **logically equivalent**
- ▶ contraposition frequently used in **indirect proofs**, as it is often easier to show than original implication

contraposition

$$\frac{X_0 \implies X_1}{\neg X_1 \implies \neg X_0}$$

example

"If there is a burglar, the dog barks."

This is logically equivalent to the statement:

"If the dog does not bark, there is no burglar."

Notes:

Group Exercise 07-01

- Use the rule of contraposition to prove the following proposition about an integer number x .

If x^2 is uneven, then x is uneven.

We could use modus ponens to make a **direct proof** of $X_0 \implies X_1$, i.e. starting from hypothesis X_0 (x^2 is uneven) we show that X_1 (x is uneven) follows.

Using contraposition, we can instead prove the **logically equivalent proposition** $\neg X_1 \implies \neg X_0$, i.e. if x is even, then x^2 is even.

Proof: Let us assume $\neg X_0$, i.e. x is even.

If x is even, there exists a $k \geq 1$ with $x = 2 \cdot k$.

We then have $x^2 = 2 \cdot k \cdot 2 \cdot k$ and thus $x^2 = 2 \cdot (2k^2)$, i.e. x^2 is even and thus $\neg X_1$.

Due to $(\neg X_1 \implies \neg X_0) \equiv (X_0 \implies X_1)$ we thus have $X_0 \implies X_1$ □

Notes:

Reductio ad absurdum

- ▶ reductio ad absurdum (“reduce to the absurd”) indirectly proves proposition by showing that its negation would lead to **contradiction**

- ▶ we can use truth table to prove **underlying tautology**

		$A :=$	$B :=$	
X_0	X_1	$X_0 \implies X_1$	$X_0 \implies \neg X_1$	$A \wedge B \implies \neg X_0$
0	0	1	1	1
0	1	1	1	1
1	0	0	1	1
1	1	1	0	1

- ▶ can be alternatively expressed with tautology

$$(X_0 \implies X_1) \iff ((X_0 \wedge \neg X_1) \implies \text{False})$$

- ▶ **proof by contradiction** frequently used in mathematics

reductio ad absurdum

$$\frac{X_0 \implies X_1 \quad X_0 \implies \neg X_1}{\neg X_0}$$

example

“Whenever there is a party, I definitely go out. At the same time, whenever there is a party, I definitely stay at home. Then there cannot be a party.”

Notes:

Group Exercise 07-02

- Use the reductio ad absurdum rule to prove the following proposition about two integer numbers x and y .

If x and y are even, then $x \cdot y$ is even.

We could make a **direct proof** of $X_0 \implies X_1$, i.e. starting from hypothesis X_0 (x and y are even) we show that X_1 ($x \cdot y$ is even) follows. Using reductio ad absurdum, we can instead show the logically equivalent proposition $(X_0 \wedge \neg X_1) \implies \text{False}$.

Proof: Let us assume X_0 and $\neg X_1$, i.e. x and y are even while $x \cdot y$ is uneven.

If x is even, there exists a $k_1 \geq 1$ with $x = 2 \cdot k_1$.

If y is even, there exists a $k_2 \geq 1$ with $y = 2 \cdot k_2$.

We have $x \cdot y = 2 \cdot k_1 \cdot 2 \cdot k_2 = 2 \cdot k'$. This implies that $x \cdot y$ is even.

The conjunction X_0 (x and y are even) and $\neg X_1$ ($x \cdot y$ is uneven) thus yields false.

Due to $(X_0 \wedge \neg X_1) \implies \text{False} \equiv (X_0 \implies X_1)$ we thus have $X_0 \implies X_1$ □

Notes:

A simple logical inference algorithm

- ▶ consider a **knowledgebase** KB captured in propositional logic, e.g.

$$(X_0 \iff (X_1 \vee X_2)) \wedge \neg X_0$$

- ▶ can we use this to automatically prove the correctness of a given **theorem** α , e.g.

$$\neg X_1$$

- ▶ simple idea: **enumerate truth table** and show that $KB \implies \alpha$ is tautology, i.e.

$$((X_0 \iff (X_1 \vee X_2)) \wedge \neg X_0) \implies \neg X_1$$

- ▶ this is necessarily correct and works for any knowledgebase and theorem, i.e. **logical inference in propositional logic is decidable**
- ▶ but: complexity of **truth table enumeration** is in $2^{O(n)}$ where n is number of propositional variables X_i

Notes:

Forward reasoning

- ▶ to prove $KB \implies \alpha$ it is sufficient to **check that α is true whenever KB is true**
- ▶ we only need to **check rows where KB is true**
- ▶ idea: use search algorithm that repeatedly **applies rules of inference** to rows where knowledgebase is **True**
 - ▶ pick rule of inference where premises match knowledgebase
 - ▶ add conclusion to knowledgebase
 - ▶ repeat until α is in knowledgebase (or we can generate no new conclusions)
- ▶ this inference method is called **forward chaining** or **forward reasoning**
- ▶ basis for inference engines like CLIPS

$$KB := (X_0 \iff (X_1 \vee X_2)) \wedge \neg X_0$$

X_0	X_1	X_2	KB	$\neg X_1$	$KB \implies \neg X_1$
0	0	0	1	1	1
0	0	1	0	1	1
0	1	0	0	0	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	0	1	1
1	1	0	0	0	1
1	1	1	0	0	1



image credit: www.clipsrule.net

Notes:

- CLIPS = C Language Integrated Production System, developed at NASA between 1985 and 1996

Group Exercise 07-03

- Use forward chaining of rules of inference to prove the theorem $\neg X_1$ based on knowledgebase $(X_0 \iff (X_1 \vee X_2)) \wedge \neg X_0$.

1. Using $(A \iff B) \equiv (A \implies B) \wedge (B \implies A)$ we eliminate $\iff$ and obtain
$$[X_0 \implies (X_1 \vee X_2)] \wedge [(X_1 \vee X_2) \implies X_0] \wedge \neg X_0$$

2. we use $A \implies B \equiv \neg A \vee B$ to eliminate $\implies$ and get
$$[\neg X_0 \vee X_1 \vee X_2] \wedge [\neg(X_1 \vee X_2) \vee X_0] \wedge \neg X_0$$

3. De Morgan's law yields
$$[\neg X_0 \vee X_1 \vee X_2] \wedge [\neg X_1 \wedge \neg X_2 \vee X_0] \wedge \neg X_0$$

4. using the **and-elimination rule**, we can infer
$$[\neg X_1 \wedge \neg X_2 \vee X_0] \wedge \neg X_0$$

5. we apply the **disjunctive syllogism rule** and infer
$$\neg X_1 \wedge \neg X_2$$

6. a final application of the **and-elimination rule** allows us to infer the theorem
$$\neg X_1 \quad \square$$

Notes:

Reconsidering modus ponens

- ▶ using equivalence $X_0 \implies X_1 \equiv \neg X_0 \vee X_1$ we can rewrite modus ponens rule

$$X_0 \wedge (\neg X_0 \vee X_1) \implies X_1$$

- ▶ CNF, where **one disjunction contains complement of variable** (X_0) that also appears in other disjunction
- ▶ conclusion contains variables that
 - (i) appear in one of the two disjunctions
 - (ii) do not appear as complement of X_0 in other disjunction
- ▶ this is a trivial case of **resolution inference rule**

modus ponens

$$\frac{X_0 \implies X_1 \quad X_0}{X_1}$$

modus ponens (rewritten)

$$\frac{X_0 \quad \neg X_0 \vee X_1}{X_1}$$

Notes:

Resolution

- consider the following formula in CNF

$$(X_0 \vee \dots \vee X_k \vee C) \wedge (Y_0 \vee \dots \vee Y_l \vee \neg C)$$

- this implies $X_0 \vee \dots \vee X_k \vee Y_0 \vee \dots \vee Y_l$

X_0	Y_0	C	$A := (X_0 \vee C) \wedge (Y_0 \vee \neg C)$	$B := X_0 \vee Y_0$	$A \Rightarrow B$
0	0	0	0	0	1
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	1	1	1
1	0	0	1	1	1
1	0	1	0	1	1
1	1	0	1	1	1
1	1	1	1	1	1

- resolution** allows to infer new formula in which we eliminate variables appearing with their complement
- for special case $k = l = 0$ we have contradiction and **resolution yields empty clause**

resolution

$$\frac{X_0 \vee \dots \vee X_k \vee C \quad Y_0 \vee \dots \vee Y_l \vee \neg C}{X_0 \vee \dots \vee X_k \vee Y_0 \vee \dots \vee Y_l}$$

resolution to empty clause

$$\frac{C \quad \neg C}{\{\}}$$

Notes:

Resolution and satisfiability

- ▶ we can use resolution to **check satisfiability** of formula
- ▶ consider **formula** $F = c_1 \wedge \dots \wedge c_k$ **in CNF** as set of clauses c_i

$$\{c_1, c_2, \dots, c_k\}$$

- ▶ **apply resolution rule** to all pairs of clauses c_i, c_j that contain complementary variables
 1. add resolved clause r to formula
 2. remove clauses c_i, c_j from formula
- ▶ repeat until we get **empty clause** r (unsatisfiable) or cannot apply resolution (satisfiable)
- ▶ DPLL is based on **resolution-based search algorithm** proposed in → M Davis and H Putnam, 1960

$$(X_0 \vee X_1 \vee X_2) \wedge (\neg X_1 \vee \neg X_2) \wedge X_0 \wedge \neg X_0$$

resolution algorithm

```
procedure RESOLUTION( $F$ )  
  choose  $X$  that also appears as  $\neg X$  in  $F$   
  for clauses  $c_i$  with  $X \in c_i$  do  
    for clauses  $c_j$  with  $\neg X \in c_j$  do  
       $r \leftarrow c_i - \{X\} \vee c_j - \{\neg X\}$   
       $F \leftarrow \text{add\_clause}(F, r)$   
    end for  
  end for  
  for clauses  $c_i$  with  $X \in c_i$  or  $\neg X \in c_i$  do  
     $F \leftarrow \text{remove\_clause}(F, c_i)$   
  end for  
end procedure
```

Notes:

Proving propositional theorems

- ▶ consider **knowledgebase** captured by expression in propositional logic, e.g.

$$(X_0 \iff (X_1 \vee X_2)) \wedge \neg X_0$$

- ▶ to **prove theorem** $\alpha = \neg X_1$ let us add a conjunction with its complement $\neg\alpha$, i.e.

$$(X_0 \iff (X_1 \vee X_2)) \wedge \neg X_0 \wedge X_1$$

- ▶ applying **reductio ad absurdum**, we can prove theorem by showing that resulting expression always evaluates to False, i.e. $KB \wedge \neg\alpha$ **is unsatisfiable**
- ▶ **resolution is complete**, i.e. resolution alone (and thus DPLL) is sufficient to prove satisfiability of **any propositional formula**

theorem proving algorithm

1. append complement of theorem α to knowledgebase
2. transform formula to CNF (eliminating $\implies$ and $\iff$)
3. run DPLL to check satisfiability
4. unsatisfiability proves theorem α

Notes:

Group Exercise 07-04

- Use resolution to prove theorem $\neg X_1$ based on knowledgebase $X_0 \iff (X_1 \vee X_2) \wedge \neg X_0$

1. we eliminate $\iff$ and $\implies$ to bring formula in CNF → group exercise 07-03

$$(\neg X_0 \vee X_1 \vee X_2) \wedge (X_0 \vee \neg X_1) \wedge (X_0 \vee \neg X_2) \wedge \neg X_0$$

2. we add complement X_1 of our theorem $\neg X_1$, i.e.

$$(\neg X_0 \vee X_1 \vee X_2) \wedge (X_0 \vee \neg X_1) \wedge (X_0 \vee \neg X_2) \wedge \neg X_0 \wedge X_1$$

3. we choose X_0 and apply resolution rule

$$(X_1 \vee \neg X_1 \vee X_2) \wedge (X_1 \vee X_2 \vee \neg X_2) \wedge \neg X_1 \wedge \neg X_2 \wedge X_1$$

4. choosing X_1 and applying resolution to $\{\neg X_1\}$ and $\{X_1\}$ yields empty clause.

5. Since $KB \implies \alpha$ unsatisfiable, we have proven $\neg X_1$

□.

Notes:

Practice Session

- ▶ we implement the **truth table enumeration algorithm**
- ▶ we implement a **DPLL-based theorem prover**
- ▶ we use both approaches to prove theorem $\neg X_1$ given knowledgebase $KB := (X_0 \iff (X_1 \vee X_2)) \wedge \neg X_0$.

X_0	X_1	X_2	KB	$\neg X_1$	$KB \implies \neg X_1$
0	0	0	1	1	1
0	0	1	0	1	1
0	1	0	0	0	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	0	1	1
1	1	0	0	0	1
1	1	1	0	0	1

05-01 Logical Inference

May 3, 2023
Ingo Scholtes

```
1 import pandas as pd
2 import itertools

3
4 def implies(x, y):
5     return not(x) or y
6
7 def biconditional(x, y):
8     return x == y
9
10
11 def check_implication(knowledge, query, variables):
12     # generate truth table
13     tt = []
14     for assignment in itertools.product([False, True], repeat=len(variables)):
15         row = [x for x in assignment]
16         row.append(knowledge(*assignment))
17         row.append(query(*assignment))
18         row.append(implies(knowledge(*assignment), query(*assignment)))
19         tt.append(row)
20     labels = variables
21     labels.append('kb')
22     labels.append('query')
23     labels.append('implication')
24     tt = pd.DataFrame(tt, columns=labels)
25     print(tt)
26
27     # check whether the implication holds
28     return tt['implication'].all()
29
30
31 check_implication(knowledge = lambda x0, x1: implies(x0, x1) and not x0,
32                  query = lambda x0, x1: x1,
33                  variables=['x0', 'x1'])
```

practice session

see notebooks 07-01 and 07-02 in gitlab repository at

→ https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2

Notes:

In summary ...

- ▶ we motivated **logical inference** in symbolic AI and covered **rules of inference** in propositional logic
- ▶ we introduced a simple inference algorithm based on **forward-chaining of rules of inference**
- ▶ we introduced **resolution**, which can be used to **prove any propositional theorem** that logically follows from its premises
- ▶ we used **DPLL to prove propositional theorems** based on a knowledgebase



DALL-E image generated for prompt “an image that represents the relation between formal logics and symbolic artificial intelligence”

image credit: www.bing.com

Notes:

Self-study questions

1. Use a truth table to show that the reductio ad absurdum rule can be expressed by the tautology $(X_0 \implies X_1) \iff ((X_0 \wedge \neg X_1) \implies \text{False})$.
2. Use De Morgan's rule to derive the conjunctive syllogism inference rule.
3. Use truth table enumeration to prove theorem X_1 based on knowledgebase $X_0 \iff ((\neg X_0 \vee X_1 \vee X_2) \wedge \neg X_0)$.
4. Use arbitrary rules of inference to prove the theorem above.
5. Use resolution to check the satisfiability of $F = (X_0 \vee X_1 \implies X_2) \iff (X_2 \vee X_0) \wedge \neg X_1$.
6. The ground resolution theorem states that if a set of clauses in CNF is unsatisfiable, then the resolution of those clauses yields an empty clause. Use contraposition to prove this theorem.
7. Explain why we can prove the theorem $KB \implies \alpha$ by showing that $KB \wedge \neg \alpha$ is unsatisfiable.

Notes:

References

reading list

- ▶ M Davis and H Putnam: **A Computing Procedure for Quantification Theory**. Journal of the ACM. 7 (3), 1960
- ▶ M Davis, G Logemann, D Loveland: **A Machine Program for Theorem Proving**, Communications of the ACM. 5 (7), 1961
- ▶ B Nebel: **Logics for Knowledge Representation**, Int. Encyc. Social and Behavioral Sciences, 2001
- ▶ S Hedman: **A First Course in Logic**, Oxford Texts in Logic, 2004
- ▶ U Schöning: **Logik für Informatiker**, Spektrum Akad. Verlag, 2005
- ▶ DE Knuth: **The Art of Computer Programming 4A: Combinatorial Algorithms**, chapter 7
- ▶ S Russel and P Norvig: **Artificial Intelligence: A modern Approach**, 2022, chapter 7

acknowledgments

examples partly based on the following books and lectures:

- ▶ V Goranko: **First-Order Logic: Satisfiability, validity, logical consequence**, DTU, 2010
- ▶ C Meinel, M Mundhenk: **Mathematische Grundlagen der Informatik**, Springer, 2014
- ▶ S Russel and P Norvig: **Artificial Intelligence: A modern Approach**, 2022 → section 7.5

A Computing Procedure for Quantification Theory*

MARTIN DAVIS

Researcher Polytechnic Institute, Hartford Division, East Windsor Hill, Conn.

AND

HILARY PUTNAM

Princeton University, Princeton, New Jersey

The hope that mathematical methods employed in the investigation of formal logic would lead to purely computational methods for obtaining mathematical theorems goes back to Leibniz and has been revived by Peano around the turn of the century and by Hilbert's school in the 1920's. Hilbert, noting that all of classical mathematics could be formalized within quantification theory, declared that the problem of finding an algorithm for determining whether or not a given formula of quantification theory is valid was the central problem of mathematical logic. And indeed, at one time it seemed as if investigations of this "decision" problem were on the verge of success. However, it was shown by Church and by Turing that such an algorithm can not exist. This result led to considerable pessimism regarding the possibility of using modern digital computers in deciding significant mathematical questions. However, recently there has been a revival of interest in the whole question. Specifically, it has been realized that while no decision procedure exists for quantification theory there are many proof procedures available—that is, uniform procedures which will ultimately locate a proof for any formula of quantification theory which is valid but which will usually involve seeking "forever" in the case of a formula which is not valid—and that some of these proof procedures could well turn out to be feasible for use with modern computing machinery.

Hao Wang [9] and P. C. Gilmore [3] have each produced working programs which employ proof procedures in quantification theory. Gilmore's program employs a form of a basic theorem of mathematical logic due to Herbrand, and Wang's makes use of a formulation of quantification theory related to those studied by Gentzen. However, both programs encounter decisive difficulties with any but the simplest formulas of quantification theory, in connection with methods of doing propositional calculus. Wang's program, because of its use of Gentzen-like methods, involves exponentiation on the total number of truth-functional connectives, whereas Gilmore's program, using normal forms, involves exponentiation on the number of clauses present. Both methods are superior in many cases to truth table methods which involve exponentiation on the

* Received September, 1959. This research was supported by the United States Air Force through the Air Force Office of Scientific Research and Development Command, under Contract No. AF 49(638)-527. Reproduction in whole or in part is permitted for any purpose of the United States Government.

Notes: